

9.6 容器适配器

除了顺序容器外，标准库还定义了三个顺序容器适配器：stack、queue 和 priority_queue。适配器 (adaptor) 是标准库中的一个通用概念。容器、迭代器和函数都有适配器。本质上，一个适配器是一种机制，能使某种事物的行为看起来像另外一种事物一样。一个容器适配器接受一种已有的容器类型，使其行为看起来像一种不同的类型。例如，stack 适配器接受一个顺序容器（除 array 或 forward_list 外），并使其操作起来像一个 stack 一样。表 9.17 列出了所有容器适配器都支持的操作和类型。

369



表 9.17: 所有容器适配器都支持的操作和类型	
size_type	一种类型，足以保存当前类型的最大对象的大小
value_type	元素类型
container_type	实现适配器的底层容器类型
A a;	创建一个名为 a 的空适配器
A a(c);	创建一个名为 a 的适配器，带有容器 c 的一个拷贝
关系运算符	每个适配器都支持所有关系运算符： <code>==</code> 、 <code>!=</code> 、 <code><</code> 、 <code><=</code> 、 <code>></code> 和 <code>>=</code> 这些运算符返回底层容器的比较结果
a.empty()	若 a 包含任何元素，返回 false，否则返回 true
a.size()	返回 a 中的元素数目
swap(a,b)	交换 a 和 b 的内容，a 和 b 必须有相同类型，包括底层容器类型也必须相同
a.swap(b)	

定义一个适配器

每个适配器都定义两个构造函数：默认构造函数创建一个空对象，接受一个容器的构造函数拷贝该容器来初始化适配器。例如，假定 `deq` 是一个 `deque<int>`，我们可以用 `deq` 来初始化一个新的 `stack`，如下所示：

```
stack<int> stk(deq); // 从 deq 拷贝元素到 stk
```

默认情况下，stack 和 queue 是基于 deque 实现的，priority_queue 是在 vector 之上实现的。我们可以在创建一个适配器时将一个命名的顺序容器作为第二个类型参数，来重载默认容器类型。

```
// 在 vector 上实现的空栈
stack<string, vector<string>> str_stk;
// str_stk2 在 vector 上实现，初始化时保存 svec 的拷贝
stack<string, vector<string>> str_stk2(svec);
```

对于一个给定的适配器，可以使用哪些容器是有限制的。所有适配器都要求容器具有添加和删除元素的能力。因此，适配器不能构造在 array 之上。类似的，我们也不能用 forward_list 来构造适配器，因为所有适配器都要求容器具有添加、删除以及访问尾元素的能力。stack 只要求 push_back、pop_back 和 back 操作，因此可以使用除 array 和 forward_list 之外的任何容器类型来构造 stack。queue 适配器要求 back、push_back、front 和 push_front，因此它可以构造于 list 或 deque 之上，但不能基于 vector 构造。priority_queue 除了 front、push_back 和 pop_back 操作之外还要求随机访问能力，因此它可以构造于 vector 或 deque 之上，但不能基于 list 构造。

370

栈适配器

stack 类型定义在 stack 头文件中。表 9.18 列出了 stack 所支持的操作。下面的程序展示了如何使用 stack:

```
stack<int> intStack; // 空栈
// 填满栈
for (size_t ix = 0; ix != 10; ++ix)
    intStack.push(ix);          // intStack 保存 0 到 9 十个数
while (!intStack.empty()) {    // intStack 中有值就继续循环
    int value = intStack.top();
    // 使用栈顶值的代码
    intStack.pop(); // 弹出栈顶元素, 继续循环
}
```

其中, 声明语句

```
stack<int> intStack; // 空栈
```

定义了一个保存整型元素的栈 intStack, 初始时空。for 循环将 10 个元素添加到栈中, 这些元素被初始化为从 0 开始连续的整数。while 循环遍历整个 stack, 获取 top 值, 将其从栈中弹出, 直至栈空。

表 9.18: 表 9.17 未列出的栈操作	
栈默认基于 deque 实现, 也可以在 list 或 vector 之上实现。	
s.pop()	删除栈顶元素, 但不返回该元素值
s.push(item)	创建一个新元素压入栈顶, 该元素通过拷贝或移动 item 而来, 或者
s.emplace(args)	由 args 构造
s.top()	返回栈顶元素, 但不将元素弹出栈

371

每个容器适配器都基于底层容器类型的操作定义了自己的特殊操作。我们只可以使用适配器操作, 而不能使用底层容器类型的操作。例如,

```
intStack.push(ix); // intStack 保存 0 到 9 十个数
```

此语句试图在 intStack 的底层 deque 对象上调用 push_back。虽然 stack 是基于 deque 实现的, 但我们不能直接使用 deque 操作。不能在一个 stack 上调用 push_back, 而必须使用 stack 自己的操作——push。

队列适配器

queue 和 priority_queue 适配器定义在 queue 头文件中。表 9.19 列出了它们所支持的操作。

表 9.19: 表 9.17 未列出的 queue 和 priority_queue 操作	
queue 默认基于 deque 实现, priority_queue 默认基于 vector 实现;	
queue 也可以用 list 或 vector 实现, priority_queue 也可以用 deque 实现。	
q.pop()	返回 queue 的首元素或 priority_queue 的最高优先级的元素, 但不删除此元素
q.front()	返回首元素或尾元素, 但不删除此元素
q.back()	只适用于 queue

续表

<code>q.top()</code>	返回最高优先级元素，但不删除该元素 只适用于 <code>priority_queue</code>
<code>q.push(item)</code>	在 <code>queue</code> 末尾或 <code>priority_queue</code> 中恰当的位置创建一个元素，
<code>q.emplace(args)</code>	其值为 <code>item</code> ，或者由 <code>args</code> 构造

标准库 `queue` 使用一种先进先出（`first-in, first-out, FIFO`）的存储和访问策略。进入队列的对象被放置到队尾，而离开队列的对象则从队首删除。饭店按客人到达的顺序来为他们安排座位，就是一个先进先出队列的例子。

`priority_queue` 允许我们为队列中的元素建立优先级。新加入的元素会排在所有优先级比它低的已有元素之前。饭店按照客人预定时间而不是到来时间的早晚来为他们安排座位，就是一个优先队列的例子。默认情况下，标准库在元素类型上使用 `<` 运算符来确定相对优先级。我们将在 11.2.2 节（第 378 页）学习如何重载这个默认设置。

9.6 节练习

练习 9.52: 使用 `stack` 处理括号化的表达式。当你看到一个左括号，将其记录下来。当你在一个左括号之后看到一个右括号，从 `stack` 中 `pop` 对象，直至遇到左括号，将左括号也一起弹出栈。然后将一个值（括号内的运算结果）`push` 到栈中，表示一个括号化的（子）表达式已经处理完毕，被其运算结果所替代。

小结

标准库容器是模板类型，用来保存给定类型的对象。在一个顺序容器中，元素是按顺序存放的，通过位置来访问。顺序容器有公共的标准接口：如果两个顺序容器都提供一个特定的操作，那么这个操作在两个容器中具有相同的接口和含义。

所有容器（除 array 外）都提供高效的动态内存管理。我们可以向容器中添加元素，而不必担心元素存储在哪里。容器负责管理自身的存储。vector 和 string 都提供更细致的内存管理控制，这是通过它们的 reserve 和 capacity 成员函数来实现的。

很大程度上，容器只定义了极少的操作。每个容器都定义了构造函数、添加和删除元素的操作、确定容器大小的操作以及返回指向特定元素的迭代器的操作。其他一些有用的操作，如排序或搜索，并不是由容器类型定义的，而是由标准库算法实现的，我们将在第 10 章介绍这些内容。

当我们使用添加和删除元素的容器操作时，必须注意这些操作可能使指向容器中元素的迭代器、指针或引用失效。很多会使迭代器失效的操作，如 insert 和 erase，都会返回一个新的迭代器，来帮助程序员维护容器中的位置。如果循环程序中使用了改变容器大小的操作，就要尤其小心其中迭代器、指针和引用的使用。

术语表

适配器 (adaptor) 标准库类型、函数或迭代器，它们接受一个类型、函数或迭代器，使其行为像另外一个类型、函数或迭代器一样。标准库提供了三种顺序容器适配器：stack、queue 和 priority_queue。每个适配器都在其底层顺序容器类型之上定义了一个新的接口。

数组 (array) 固定大小的顺序容器。为了定义一个 array，除了元素类型之外还必须给定大小。array 中的元素可以用其位置下标来访问。array 支持快速的随机访问。

begin 容器操作，返回一个指向容器首元素的迭代器，如果容器为空，则返回尾后迭代器。是否返回 const 迭代器依赖于容器的类型。

cbegin 容器操作，返回一个指向容器首元素的 const_iterator，如果容器为空，则返回尾后迭代器。

cend 容器操作，返回一个指向容器尾元素之后（不存在的）的 const_iterator。

容器 (container) 保存一组给定类型对象的类型。每个标准库容器类型都是一个模板类型。为了定义一个容器，我们必须指定保存在容器中的元素的类型。除了 array 之外，标准库容器都是大小可变的。

deque 顺序容器。deque 中的元素可以通过位置下标来访问。支持快速的随机访问。deque 各方面都与 vector 类似，唯一的差别是，deque 支持在容器头尾位置的快速插入和删除，而且在两端插入或删除元素都不会导致重新分配空间。

end 容器操作，返回一个指向容器尾元素之后（不存在的）元素的迭代器。是否返回 const 迭代器依赖于容器的类型。

forward_list 顺序容器，表示一个单向链表。forward_list 中的元素只能顺序访问。从一个给定元素开始，为了访问另一个元素，我们只能遍历两者之间的所有元素。forward_list 上的迭代器不支持递减运算 (--)。forward_list 支持任意位置的快速插入（或删除）操作。与其他容器不同，插入和删除发生在一个给定的

373 通过位置下标来访问。支持快速的随机访问。